

Spark SQL: Takeaways

by Dataquest Labs, Inc.

Syntax

- Registering a DataFrame as a temporary view:

```
df.createOrReplaceTempView("census2010")
```

- Running SQL queries:

```
spark.sql("SELECT age, total FROM census2010").show()
```

 - Creating calculated columns with SQL

expressions:

```
SELECT females, males, (females - males) AS gender_gap, ROUND(females * 100.0 / total, 2) AS female_percentage FROM census2010
```

 - Filtering with WHERE and multiple conditions:

```
SELECT age, total FROM census2010 WHERE age BETWEEN 13 AND 19 OR total > 4000000
```

 - Sorting

results:

```
SELECT age, total FROM census2010 ORDER BY total DESC
```

 - Aggregating data:

```
SELECT SUM(total) AS total_population, AVG(males) AS avg_males, COUNT(*) AS age_groups FROM census2010
```

 - Grouping and filtering grouped results:

```
SELECT year, SUM(total) AS total_population FROM census_all_decades GROUP BY year HAVING SUM(total) > 200000000 ORDER BY year
```

 - Combining datasets:

```
SELECT * FROM census1980 UNION ALL SELECT * FROM census2000
```

Concepts

- **Spark SQL** provides familiar SQL syntax for querying distributed DataFrames with full Catalyst optimization.
- **Temporary views** give DataFrames table-like names that can be referenced in SQL queries within your current session.
- **Global temporary views** persist across multiple sessions and are accessed through the `global_temp` namespace.
- Both **SQL queries and DataFrame operations** compile to identical execution plans, so performance is equivalent.
- **SQL excels** for complex business logic, reporting, and team collaboration, while **DataFrames work better** for programmatic transformations and Python integration.
- **Hybrid approaches** that combine both SQL and DataFrame operations are common in production workflows.

Resources

- [Spark SQL Programming Guide](#)
- [PySpark SQL API Documentation](#)
- [Spark SQL Built-in Functions](#)
- [Temporary and Global Views Documentation](#)
- [U.S. Census Data](#)

